Contents

Object Oriented Programming Languages

Complete Topic List

1. Classes and Objects

- Classes and objects, instantiation
- Comparing and copying of objects
- Testing

2. Encapsulation and Members

- Encapsulation, members, and constructors
- Instance and static members
- Information hiding and visibility modifiers

3. Polymorphism and Type Systems

- Overloading
- Inheritance and multiple inheritance
- Subtyping, static and dynamic types, and type checking
- Overriding and dynamic binding
- Interfaces
- Generics
- Subtype and parametric polymorphism

4. Memory Management

- Memory management and garbage collection

5. Program Structure

- Program structure, packages, and compilation units

Chapter Files

- `chapters/chap1_classes_objects.md`
- `chapters/chap2_encapsulation_members.md`
- `chapters/chap3_polymorphism_types.md`
- `chapters/chap4_memory_management.md`
- `chapters/chap5_program_structure.md` # Chapter 1: Classes and Objects

1.1 Classes and Objects, Instantiation

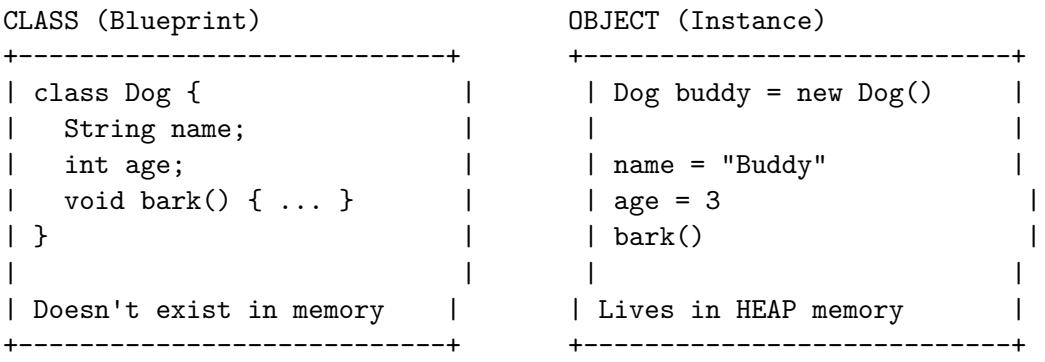
Definition / Theoretical Background

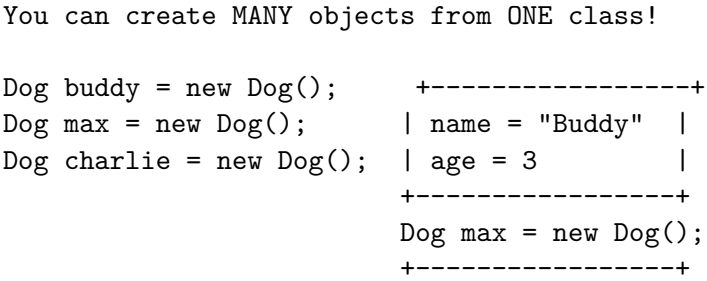
Class: A blueprint or template that defines the structure and behavior of objects. It specifies: - **Fields** (attributes/properties): Data that objects store - **Methods** (behaviors): Operations that objects can perform

Object: An instance of a class - a concrete entity created from the blueprint.

Concept	What It Is	Example
Class	Blueprint	`class Dog { ... }`
Object	Instance of class	`Dog buddy = new Dog();`
Instantiation	Creating object from class	`new Dog()`
State	Values in object's fields	`buddy.name = "Buddy"`
Identity	Unique identifier	Each object has unique address
Behavior	Methods object can perform	`buddy.bark()`

Class vs Object Visualization





```
| name = "Max"    |
| age = 5         |
+-----------------+
Dog charlie = new Dog();
+-----------------+
| name = "Charlie"|
| age = 2         |
+-----------------+
```

Declaration and Instantiation

```java
// Class definition
class Dog {
    // Instance variables (fields)
    String name;
    int age;

    // Constructor
    Dog(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Methods
    void bark() {
        System.out.println(name + " says Woof!");
    }
}

// Instantiation and use
public class Main {
    public static void main(String[] args) {
        Dog buddy = new Dog("Buddy", 3);  // Create object
        Dog max = new Dog("Max", 5);

        buddy.bark();  // "Buddy says Woof!"
        max.bark();    // "Max says Woof!"

        // Objects have independent state
        buddy.age = 4;  // Only buddy's age changes
        System.out.println(max.age);  // Still 5!
    }
}
```

Java

```python
# Class definition (Python)
class Dog:
    # Constructor (special method)
    def __init__(self, name, age):
        self.name = name  # self refers to the object
        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

# Instantiation
buddy = Dog("Buddy", 3)
max = Dog("Max", 5)

buddy.bark()  # "Buddy says Woof!"
max.bark()    # "Max says Woof!"
```

Python

```cpp
class Dog {
public:
    string name;   // Instance variables
    int age;

    // Constructor
    Dog(string name, int age) : name(name), age(age) {}

    void bark() {
        cout << name << " says Woof!" << endl;
    }
};

int main() {
    Dog* buddy = new Dog("Buddy", 3);  // Heap allocation
    Dog max("Max", 5);                  // Stack allocation

    buddy->bark();  // "Buddy says Woof!"
    max.bark();     // "Max says Woof!"

    delete buddy;   // Don't forget to free heap memory!
}
```

C++

The `new` Keyword (Instantiation)

```
Dog buddy = new Dog(...);
```

What `new` does: 1. Allocates memory for the object (usually on the heap) 2. Initializes fields to default values (0, null, false) 3. Calls the constructor method 4. Returns a reference to the object

```
new Dog("Buddy", 3)
      |
      v
+---------------------------------+
| 1. Allocate memory (heap)       |
| 2. Initialize fields:            |
|    name = null                   |
|    age = 0                       |
| 3. Call constructor:             |
|    name = "Buddy"                |
|    age = 3                       |
| 4. Return reference (address)    |
+---------------------------------+
```

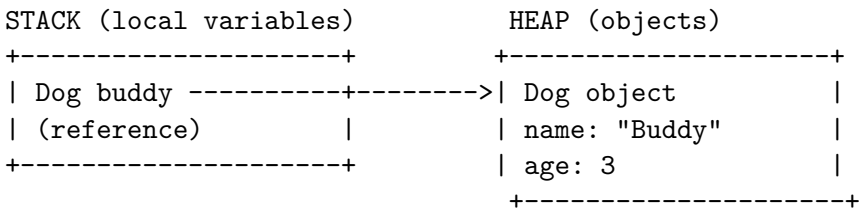
Default Values

```
class Example {
    int num;           // Default: 0
    double decimal;    // Default: 0.0
    boolean flag;      // Default: false
    String text;       // Default: null
    int[] arr;         // Default: null
    Object obj;        // Default: null

    // Constructor to initialize
    Example() {
        num = 10;  // Now it's 10
        // text still null if not initialized
    }
}
```

Default values table:

Type	Default Value
byte, short, int, long	0
float, double	0.0
char	‘0000’ (null character)
boolean	false
Object reference	null

Memory Where Objects Live



In Java, Python, C#: - **Objects live on the Heap** (managed memory) - **References live on the Stack** (variables point to heap)

In C++: - Objects can live on **Stack** (if declared locally) or **Heap** (if `new`)

Edge Cases & Traps

Trap 1: Reference vs Object Confusion

```
Dog d1 = new Dog("Buddy", 3);
Dog d2 = d1;  // Copies REFERENCE, not object!

// Now d1 and d2 point to SAME object!
d2.name = "Max";
System.out.println(d1.name);  // "Max"! Changed because same object!

// For independent copies, need to copy explicitly:
Dog d3 = new Dog(d1.name, d1.age);  // Creates NEW object
```

Trap 2: Null Reference

```
Dog d;  // d is null (no object!)
d.bark();  // NullPointerException! Object doesn't exist!

// Must initialize first:
d = new Dog("Buddy", 3);
d.bark();  // Now works
```

Trap 3: Uninitialized Fields

```
class Example {
    int x;  // Default 0, not uninitialized garbage!
}
// x is 0, not random garbage
// Unlike C/C++ local variables!
```

Trap 4: Class is a Type, Object is an Instance

```
Dog d = new Dog();  // Dog is type/class, d is instance
// You cannot do: new Dog.bark()  (wrong!)
// Must: d.bark() (using object instance)
```

Examples

```java
class BankAccount {
    // Fields
    String accountNumber;
    String ownerName;
    double balance;

    // Constructor
    BankAccount(String accNum, String name, double initialBalance) {
        accountNumber = accNum;
        ownerName = name;
        balance = initialBalance;
    }

    // Methods
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        } else {
            System.out.println("Insufficient funds!");
        }
    }

    void display() {
        System.out.println(ownerName + ": $" + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc1 = new BankAccount("12345", "Alice", 1000);
        BankAccount acc2 = new BankAccount("67890", "Bob", 500);

        acc1.deposit(500);   // Alice: $1500
        acc2.withdraw(100);  // Bob: $400
        acc1.display();       // Alice: $1500.0
        acc2.display();       // Bob: $400.0

        // Each object has its OWN state
        System.out.println(acc1.balance);  // 1500.0
        System.out.println(acc2.balance);  // 400.0
```

```
    }
}
```

Example 1: Bank Account Class

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def scale(self, factor):
        self.width *= factor
        self.height *= factor

# Create two rectangle objects
r1 = Rectangle(5, 3)
r2 = Rectangle(4, 2)

print(f"r1 area: {r1.area()}")  # 15
print(f"r2 area: {r2.area()}")  # 8

r1.scale(2)  # Scale r1 by 2
print(f"r1 area after scale: {r1.area()}")  # 60
print(f"r2 unchanged: {r2.area()}")  # Still 8!
```

Example 2: Rectangle Class

```
class Student {
    String name;
    int age;
    String major;

    // Constructor 1: Full
    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
        this.major = major;
    }

    // Constructor 2: Default major
```

```
    Student(String name, int age) {
        this.name = name;
        this.age = age;
        this.major = "Undeclared";
    }

    // Constructor 3: Default values
    Student() {
        this.name = "Unknown";
        this.age = 0;
        this.major = "Undeclared";
    }
}

// Usage
Student s1 = new Student("Alice", 20, "CS");          // Full
Student s2 = new Student("Bob", 19);                   // Default major
Student s3 = new Student();                            // All defaults
```

Example 3: Constructor Overloading (Multiple Constructors)

The `this` Reference

The `this` keyword is a reference to the **current object** - the object on which the method is being called.

```
class Person {
    private String name;
    private int age;

    // this helps distinguish field from parameter
    Person(String name, int age) {
        this.name = name;   // this.name = field, name = parameter
        this.age = age;     // this.age = field, age = parameter
    }

    // this can be used to return the object itself (for chaining)
    Person setName(String name) {
        this.name = name;
        return this;  // Returns the same object
    }

    Person setAge(int age) {
        this.age = age;
        return this;
    }

    // Chain method calls
```

```
    public static void main(String[] args) {
        Person p = new Person("Alice", 20);
        p.setName("Bob").setAge(25);  // Method chaining!
    }
}
```

Common uses of `this`:

Use	Example	When Needed
Disambiguate field/parameter	`this.x = x`	Parameter has same name as field
Return current object	`return this`	For method chaining
Pass current object	`callback(this)`	When object needs to reference itself
Constructor chaining	`this(args)`	Call another constructor in same class

this() for Constructor Chaining:

```
class Student {
    String name;
    int age;
    String major;

    // Chain to the full constructor
    Student(String name, int age) {
        this(name, age, "Undeclared");  // Calls Student(String, int, String)
    }

    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
        this.major = major;
    }
}
```

Important Note: `this()` must be the **first statement** in constructor!

Immutability

An **immutable object** is an object whose state **cannot be changed** after creation.

```
// Immutable class example
final class Person {
    private final String name;  // final = can't reassign
    private final int age;

    // No setters! Only getters and constructor
    Person(String name, int age) {
        this.name = name;   // Can't change after construction
        this.age = age;
```

```
    }

    // Only getters, no setters
    public String getName() { return name; }
    public int getAge() { return age; }
}

// Usage
Person p = new Person("Alice", 25);
String n = p.getName();  // Can read, can't modify
// p.setAge(26);         // ERROR! No setter method exists!
```

Why Immutability Matters:

```
Benefits of Immutable Objects:
1. Thread-safe - No synchronization needed!
2. Cacheable - Can safely reuse instances
3. No defensive copies needed
4. Building blocks for reliable code

Examples of immutable types:
- String in Java
- int, double primitives
- LocalDate in Java 8
- tuple in Python
```

Creating Immutable Classes:

```
public final class ImmutablePoint {
    private final int x;
    private final int y;

    // No setters! Only constructor and getters
    public ImmutablePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }

    // Returns NEW point, doesn't modify this
    public ImmutablePoint translate(int dx, int dy) {
        return new ImmutablePoint(x + dx, y + dy);
    }
}

// Usage
ImmutablePoint p1 = new ImmutablePoint(1, 2);
```

```
ImmutablePoint p2 = p1.translate(3, 4);  // Creates NEW point, p1 unchanged
```

The Problem with Mutable Objects:

```
// Mutable version - problematic!
class MutablePoint {
    int x, y;
    MutablePoint(int x, int y) { this.x = x; this.y = y; }
    void setX(int x) { this.x = x; }
    void setY(int y) { this.y = y; }
}

// Problem: If stored in collection, can be modified from outside!
MutablePoint p = new MutablePoint(1, 2);
List<MutablePoint> points = new ArrayList<>();
points.add(p);
points.get(0).setX(999);  // p.x is now 999! Original modified!

// Immutable version - safe!
ImmutablePoint p1 = new ImmutablePoint(1, 2);
points.add(p1);
// No way to modify p1 after creation!
```

1.2 Comparing and Copying Objects

Comparing Objects

Reference Equality vs Value Equality

Type	What It Checks	Syntax
Reference equality	Same memory location?	== in Java for references
Value equality	Same content?	`.equals()` in Java

```
String s1 = new String("hello");
String s2 = new String("hello");

// Reference comparison (are they the same object?)
System.out.println(s1 == s2);           // false (different objects)

// Value comparison (do they have same content?)
System.out.println(s1.equals(s2));      // true (same content!)

// BUT: String literals are pooled!
String s3 = "hello";
String s4 = "hello";
```

```
System.out.println(s3 == s4);           // true (same pooled object!)
System.out.println(s3.equals(s4));     // true (same content too)
```

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);

System.out.println(p1 == p2);       // false! Different objects!
System.out.println(p1.equals(p2)); // false! equals() inherited from Object
                                    // does reference comparison by default!

// You need to OVERRIDE equals()!
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;  // Same reference
        if (!(obj instanceof Point)) return false;
        Point other = (Point) obj;
        return this.x == other.x && this.y == other.y;
    }
}
```

Why == Doesn't Work for Value Comparison

```
# Python: == does VALUE comparison by default!
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = p1

print(p1 == p2)  # True! (Python's == compares values by default)
print(p1 == p3)  # True! (Same object, same value)

# is checks identity (like reference equality in Java)
```

```
print(p1 is p2)  # False! Different objects
print(p1 is p3)  # True! Same object
```

Python Comparison

Copying Objects

```
class Address {
    String city;
    Address(String city) { this.city = city; }
}

class Person {
    String name;
    Address address;  // Reference type field!

    Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }
}

// Original
Address addr = new Address("NYC");
Person original = new Person("Alice", addr);

// Shallow copy - copies reference, not object!
Person shallowCopy = original;
// shallowCopy.address points to SAME Address object as original!

// Deep copy - copies the referenced objects too!
Person deepCopy = new Person(original.name, new Address(original.address.city));
// deepCopy.address is a NEW object with same values!
```

Shallow Copy vs Deep Copy

Shallow vs Deep Copy Visualization

```
SHALLOW COPY:
original ----------------------+
    |                          |
    | address -------------+   |
    v                      v   v
[Address: NYC]    shallowCopy.address -- same object!

DEEP COPY:
original ----------------------+
```

```
    |                          |
    | address --------------+  |   (original.address)
    v                       |  |
[Address: NYC]              |  |
                            |  |
deepCopy.address -----------+  |
    |                          |
    v                          v
[Address: NYC]              NEW COPY! Different object
```

```java
class Person {
    String name;
    int age;

    // Regular constructor
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Copy constructor
    Person(Person other) {
        this.name = other.name;
        this.age = other.age;
    }
}

// Using copy constructor
Person original = new Person("Alice", 25);
Person copy = new Person(original);  // Creates NEW object with same values!
```

Copy Constructors

```python
import copy

class Person:
    def __init__(self, name, address):
        self.name = name
        self.address = address  # Could be object reference

# Original
p1 = Person("Alice", "NYC")

# Shallow copy - nested objects shared
p2 = copy.copy(p1)
```

```python
# Deep copy - nested objects also copied
p3 = copy.deepcopy(p1)

# p1 and p2 are different objects, but their address field might be same object
# p3 is completely independent
```

Python Copy Module

Edge Cases & Traps

Trap 1: == on Objects Compares References, Not Values

```java
String a = new String("test");
String b = new String("test");
if (a == b) {  // FALSE! Different objects!
    // This block never executes!
}
if (a.equals(b)) {  // TRUE! Same content!
    // This block executes!
}
```

[WARNING] **Professor Trap**: Always use `.equals()` for string value comparison!

Trap 2: Mutable Default Arguments (Python)

```python
# DANGER! This is a common Python trap!
class BadList:
    def __init__(self, items=[]):  # Mutable default!
        self.items = items

lst1 = BadList()
lst1.items.append(1)

lst2 = BadList()
print(lst2.items)  # [1]! Shared default list!

# CORRECT:
def __init__(self, items=None):
    if items is None:
        items = []  # Create new list each time
```

Trap 3: Shallow Copy with Nested Objects

```java
class Node {
    Node next;
    int value;
}

Node n1 = new Node();
n1.value = 1;
Node n2 = new Node();
```

```
n2.value = 2;
n1.next = n2;

// Shallow copy of n1
Node copy = new Node();
copy.value = n1.value;
copy.next = n1.next;  // copy.next points to SAME n2!

// Now modifying n2 affects both original and copy!
```

Trap 4: Forgetting to Override equals()

```
class Point {
    int x, y;
    // No equals() override!
}

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);

// These are both false!
System.out.println(p1 == p2);      // false (different objects)
System.out.println(p1.equals(p2)); // false (inherited == checks references!)
```

[WARNING] **Professor Trap**: If you override equals(), also override hashCode() for Maps/Sets!

Examples

```
class Point {
    private int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;  // Same reference
        if (obj == null || getClass() != obj.getClass()) return false;
        Point other = (Point) obj;
        return x == other.x && y == other.y;
    }

    @Override
    public int hashCode() {
        return 31 * x + y;  // Consistent with equals()
    }
```

```
}

// Now:
Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);
System.out.println(p1.equals(p2));  // true! Same values!
System.out.println(p1 == p2);       // false! Different objects!
```

Example 1: Proper equals() Implementation

```
class Person implements Cloneable {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    protected Object clone() {
        return new Person(this.name, this.age);  // Deep copy
    }
}

Person original = new Person("Alice", 25);
Person cloned = (Person) original.clone();

System.out.println(original == cloned);       // false
System.out.println(original.name.equals(cloned.name));  // true
```

Example 2: Clone Method

1.3 Testing

Testing Concepts

Why test? - Find bugs before users do - Verify functionality works correctly - Enable confident refactoring - Document expected behavior

Unit Testing Basics

```
// Simple unit test example (JUnit)
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

```java
class Calculator {
    int add(int a, int b) { return a + b; }
    int divide(int a, int b) {
        if (b == 0) throw new ArithmeticException("Division by zero");
        return a / b;
    }
}

class CalculatorTest {
    @Test
    void testAdd() {
        Calculator calc = new Calculator();
        assertEquals(5, calc.add(2, 3));
        assertEquals(0, calc.add(-1, 1));
    }

    @Test
    void testDivide() {
        Calculator calc = new Calculator();
        assertEquals(2, calc.divide(10, 5));
    }

    @Test
    void testDivideByZero() {
        Calculator calc = new Calculator();
        assertThrows(ArithmeticException.class, () -> calc.divide(1, 0));
    }
}
```

Testing Terminology

Term	Meaning
Unit test	Tests single method/class
Integration test	Tests multiple components together
Test fixture	Setup code before tests
Assertion	Check that expected == actual
Mock	Fake object for testing dependencies
TDD	Test-Driven Development (write tests first)

White-Board Example: Testing OOP Concepts

```
Question: What is the output?

Dog d1 = new Dog("Buddy", 3);
Dog d2 = d1;           // What is d2?
d2.setAge(4);
```

```
System.out.println(d1.getAge());

Answer: 4

Explanation:
1. d1 created with age 3
2. d2 = d1 means d2 points to SAME object as d1
3. d2.setAge(4) modifies the shared object
4. d1.getAge() returns 4

This tests understanding of REFERENCE semantics!
```

Practice Questions

MCQ 1: What does `new` do in Java? - A) Declares a new variable - B) Allocates memory and calls constructor - C) Copies an existing object - D) Deletes an object

MCQ 2: Two objects created with `new Dog()` and same values - what is `==`? - A) true - B) false - C) Depends on equals() method - D) Compile error

MCQ 3: What is shallow copy? - A) Copies all fields including references - B) Copies only primitive fields - C) Creates new referenced objects - D) Used for value types only

Short Answer: Explain why `String s1 = new String("hi") == String s2 = new String("hi")` returns false.

Board Coding: Write a class `Circle` with radius field, constructor, and method `getArea()`. Then write tests for it.

Oral Explanation: What's the difference between `==` and `.equals()` for objects? When would you use each?

Solutions

MCQ 1: B) Allocates memory and calls constructor

MCQ 2: B) false - different objects in memory

MCQ 3: A) Copies all fields, but reference fields point to same objects

Short Answer: `new` creates a new object in heap memory. Even though both strings have content "hi", they are different objects at different memory locations, so `==` compares references (addresses) and returns false. Use `.equals()` for content comparison.

Cheat-Sheet

Concept	Key Points
Class	Blueprint/template defining structure and behavior
Object	Instance of class, lives in heap memory
Instantiation	`new ClassName()` allocates and initializes

Concept	Key Points
Constructor	Initializes new object, has same name as class
Reference	Variable that points to object in heap
null	Reference pointing to nothing
==	Compares references (memory addresses)
equals()	Compares object values (override for custom comparison)
Shallow copy	Copies object, shared references inside
Deep copy	Copies object AND all nested objects

Review Checklist

- ☐ **Class vs Object**: Can you explain the difference?
- ☐ **Instantiation**: Do you understand what `new` does?
- ☐ **Reference semantics**: Can you trace reference assignments?
- ☐ **Comparison**: Do you know when to use == vs equals()?
- ☐ **Copying**: Can you explain shallow vs deep copy?
- ☐ **Testing**: Can you write basic unit tests? # Chapter 2: Encapsulation and Members

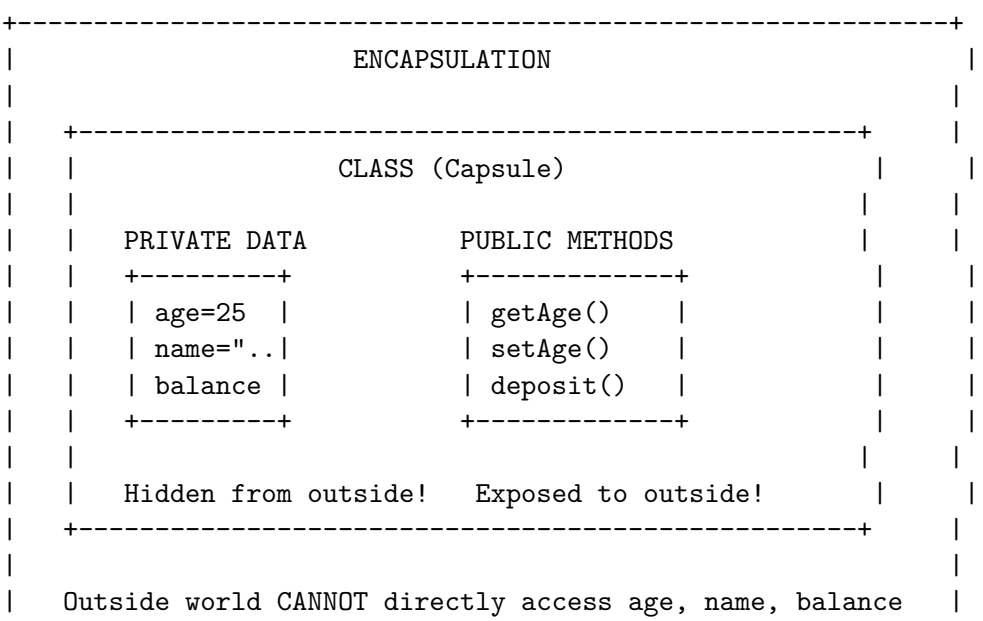
2.1 Encapsulation

Definition / Theoretical Background

Encapsulation is the bundling of data (fields) and methods (behaviors) into a single unit (class), and restricting access to some of the object's internal components.

Key Benefits: 1. **Data hiding** - Hide internal implementation details 2. **Protection** - Prevent external code from corrupting state 3. **Modularity** - Objects are self-contained, independent units 4. **Flexibility** - Can change internal implementation without affecting users

The Encapsulation Principle



```
|   They MUST go through public methods (getAge, setAge...)   |
+-------------------------------------------------------------+
```

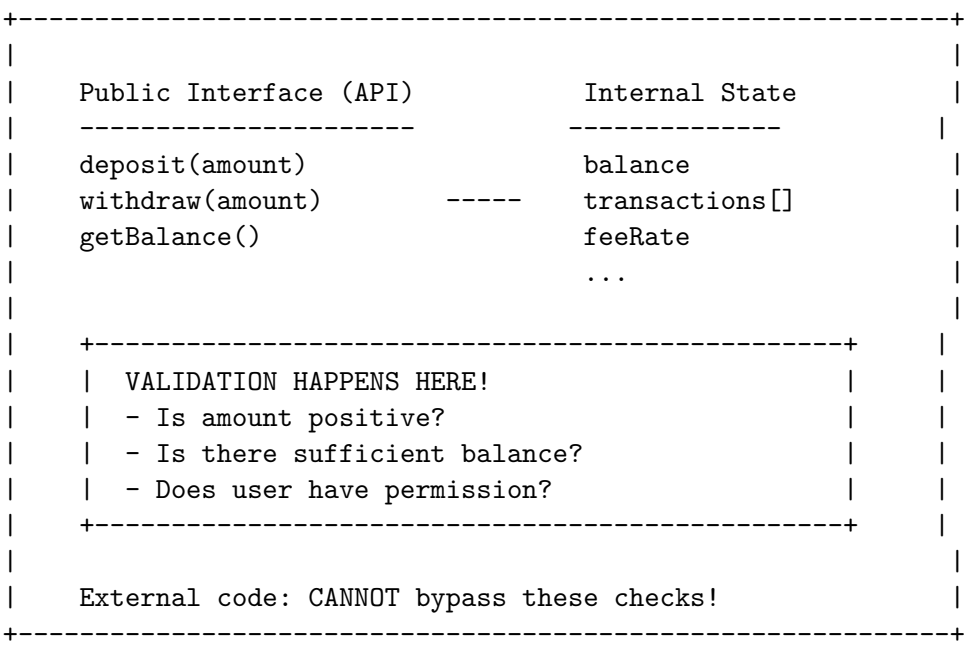
Why Encapsulation Matters

```
// WITHOUT encapsulation (BAD!)
class BankAccountBad {
    double balance;  // PUBLIC! Anyone can modify!

    BankAccountBad(double balance) {
        this.balance = balance;  // No validation!
    }
}

// Anyone can do dangerous things:
account.balance = -1000000;  // Negative balance! No validation!
account.balance = Double.MAX_VALUE;  // Overflow possible!
```

```
// WITH encapsulation (GOOD!)
class BankAccountGood {
    private double balance;  // PRIVATE!

    BankAccountGood(double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Cannot be negative!");
        }
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive!");
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive!");
        }
        if (amount > balance) {
            throw new IllegalArgumentException("Insufficient funds!");
        }
        balance -= amount;
    }

    public double getBalance() {
        return balance;  // Read-only access (no setter!)
```

```
    }
}

// NOW: Balance is protected! All operations validated!
// account.balance = -1000000;  // ERROR! Can't access private field!
// Must use: account.deposit() which validates!
```

The Protective Barrier



2.2 Members (Fields and Methods)

Instance Members vs Static Members

Aspect	Instance Member	Static Member
Belongs to	Each object (instance)	Class itself
Memory	Each object has its own copy	Single copy shared
Access	`object.member`	`ClassName.member`
Lifetime	As long as object exists	Whole program
Keyword	(none)	`static`

Instance Members

```
class Dog {
    String name;  // Instance field - each Dog has its own name
    int age;      // Instance field - each Dog has its own age
```

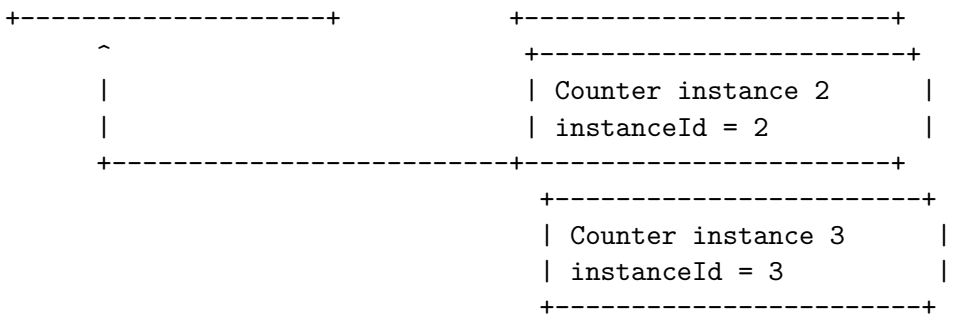
```
    void bark() {  // Instance method - operates on specific instance
        System.out.println(name + " says Woof!");
    }
}

Dog buddy = new Dog();
Dog max = new Dog();

buddy.name = "Buddy";  // buddy's name
max.name = "Max";       // max's name (independent!)

buddy.bark();  // "Buddy says Woof!" - operates on buddy
max.bark();    // "Max says Woof!" - operates on max
```

Static Members

```
class Counter {
    private static int count = 0;  // Static field - ONE copy for class
    private int instanceId;       // Instance field - one per object

    Counter() {
        instanceId = ++count;  // Increment shared count
    }

    public static int getCount() {  // Static method
        return count;  // Can only access static members
    }

    public int getInstanceId() {   // Instance method
        return instanceId;  // Can access both static and instance
    }
}

Counter c1 = new Counter();  // instanceId=1, count=1
Counter c2 = new Counter();  // instanceId=2, count=2
Counter c3 = new Counter();  // instanceId=3, count=3

// All objects share the SAME count!
System.out.println(Counter.getCount());  // 3 (via class name)
// c1.getCount() also works but convention is ClassName.method()
```

Static Field Memory Visualization

```
STATIC (class-level)              HEAP (per object)
+--------------------+           +------------------------+
| Counter.count = 3  |           | Counter instance 1     |
| (shared by all!)   |           | instanceId = 1         |
```



When to Use Static?

```
// Use STATIC when:
// 1. Value is shared by ALL instances
class MathUtils {
    static final double PI = 3.14159;  // Universal constant

    static int max(int a, int b) {    // Utility method, no instance needed
        return a > b ? a : b;
    }
}

// Use INSTANCE when:
// 1. Each object needs its own copy
class Person {
    String name;  // Each person has their own name
    int age;      // Each person has their own age
}
```

Static Constants vs Instance Constants

```
class Circle {
    // Static constant - shared by all Circle objects
    static final double PI = 3.14159;

    // Instance field - each Circle has its own radius
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double area() {
        // Can use static PI here
        return PI * radius * radius;  // PI is shared constant
    }
```

```
    public double circumference() {
        return 2 * PI * radius;
    }
}

Circle c1 = new Circle(5);
Circle c2 = new Circle(10);

c1.area();   // Uses PI=3.14159, radius=5
c2.area();   // Uses PI=3.14159, radius=10
// Both use SAME PI value (one copy in memory)
```

Edge Cases & Traps

Trap 1: Static Method Can't Access Instance Members

```
class Example {
    int x = 10;  // Instance field

    static void printX() {
        System.out.println(x);  // ERROR! Can't access instance field!
        // Because which x? There's no object context!
    }
}
```

[WARNING] **Professor Trap**: Static methods don't have `this` reference!

Trap 2: Static vs Instance Method Confusion

```
class Example {
    static int count = 0;
    int instanceCount = 0;

    static void incrementStatic() {
        count++;  // OK - static field
    }

    void incrementInstance() {
        instanceCount++;  // OK - instance field
        count++;          // OK - static field also accessible
    }
}
```

[WARNING] **Professor Trap**: Static can access static, instance can access both!

Trap 3: Modifying Static from Instance Method

```
class Counter {
    static int count = 0;
```

```
    Counter() {
        count++;  // All instances share same count!
    }
}

Counter c1 = new Counter();  // count = 1
Counter c2 = new Counter();  // count = 2
Counter c3 = new Counter();  // count = 3
// All three objects see same count = 3!
```

[WARNING] **Professor Trap**: Static fields are shared! All instances see same value!

Trap 4: New vs Old Style Constants

```
// Java pre-5 style (still works but outdated)
class Constants {
    static final int MAX_SIZE = 100;  // Uppercase with underscore
}

// Modern style: enum for related constants
enum Size { SMALL, MEDIUM, LARGE }

// Or: interface with constants (bad practice, but common)
interface Constants {
    int MAX_SIZE = 100;  // Automatically static final!
}
```

2.3 Constructors

Definition / Theoretical Background

A **constructor** is a special method that: 1. Has the **same name** as the class 2. Has **no return type** 3. Is called when creating new objects with `new` 4. Initializes the object's fields

Types of Constructors

```
class Dog {
    String name;

    // Implicit default constructor (if no other constructor defined)
    // Dog() { }  // Provided automatically if you define nothing

    // Explicit default constructor
    Dog() {
        name = "Unknown";
    }
}
```

```
Dog d = new Dog();  // Calls Dog()
System.out.println(d.name);  // "Unknown"
```

Default Constructor (No-Args)

```
class Dog {
    String name;
    int age;

    Dog(String name, int age) {
        this.name = name;   // Initialize name
        this.age = age;     // Initialize age
    }
}

Dog buddy = new Dog("Buddy", 3);  // Calls Dog(String, int)
```

Parameterized Constructor

```
class Student {
    String name;
    int age;
    String major;

    // Constructor 1: Full
    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
        this.major = major;
    }

    // Constructor 2: Two params
    Student(String name, int age) {
        this.name = name;
        this.age = age;
        this.major = "Undeclared";
    }

    // Constructor 3: One param
    Student(String name) {
        this.name = name;
        this.age = 0;
        this.major = "Undeclared";
    }
```

```
    // Constructor 4: No params
    Student() {
        this.name = "Unknown";
        this.age = 0;
        this.major = "Undeclared";
    }
}

// Usage
Student s1 = new Student("Alice", 20, "CS");
Student s2 = new Student("Bob", 19);
Student s3 = new Student("Charlie");
Student s4 = new Student();
```

Multiple Constructors (Constructor Overloading)

Constructor Chaining with `this()`

```
class Student {
    String name;
    int age;
    String major;

    // This constructor calls the full one
    Student(String name, int age) {
        this(name, age, "Undeclared");  // Chain to 3-param constructor
    }

    // This constructor calls the 2-param one
    Student(String name) {
        this(name, 0);  // Chain to 2-param constructor
    }

    // Full constructor (does actual work)
    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
        this.major = major;
    }
}
```

Constructor vs Setter

```
// Constructor: Set once, at creation
class ImmutableStudent {
    private final String name;
    private final int age;
```

```
    ImmutableStudent(String name, int age) {
        this.name = name;
        this.age = age;
        // No setters! Values never change after creation
    }
}

// vs

// Setter: Can change at any time
class MutableStudent {
    private String name;
    private int age;

    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
}

// When to use which?
// - Constructor: Values known at creation, won't change (immutable)
// - Setter: Values may change during object lifetime (mutable)
```

Private Constructor (Singleton Pattern)

```
class Singleton {
    private static Singleton instance;
    private String data;

    // Private constructor - can't call from outside!
    private Singleton() {
        data = "Important data";
    }

    // Only way to get instance
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();  // Create only once!
        }
        return instance;
    }
}

// Usage
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();
System.out.println(s1 == s2);  // TRUE! Same object!
```

Edge Cases & Traps

Trap 1: Forgetting to Initialize Fields

```
class Student {
    int age;  // Default is 0, not uninitialized!

    // Student() { }  // Default constructor, age = 0
}

// Student s = new Student();
// s.age is 0, not garbage!
```

[WARNING] **Professor Trap**: Default constructor gives default values, not garbage!

Trap 2: Constructor Throwing Exception

```
class Bad {
    Resource resource;

    Bad() throws Exception {
        resource = new Resource();
        throw new Exception("Error!");  // Resource leaked!
    }
}

// If constructor throws, object is never fully created
// Destructor won't run!
// FIX: Use try-finally or RAII
```

Trap 3: Calling Overridable Method in Constructor

```
class Parent {
    Parent() {
        doSomething();  // BAD! Could call overridden method!
    }

    void doSomething() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    int x = 5;

    @Override
    void doSomething() {
        System.out.println("Child: " + x);  // x is 0 here! Not initialized yet!
    }
}
```

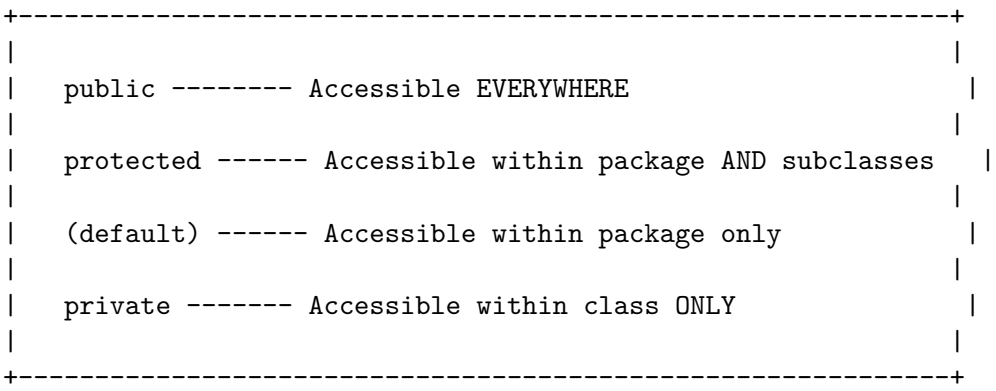
```
new Child();  // Prints "Child: 0" because child constructor hasn't run yet!
```

[WARNING] **Professor Trap**: Don't call overridable methods in constructors!

2.4 Information Hiding and Visibility Modifiers

The Four Access Levels

Modifier	Same Class	Same Package	Subclass	Anywhere
private	YES	NO	NO	NO
default (package-private)	YES	YES	NO	NO
protected	YES	YES	YES	NO
public	YES	YES	YES	YES



Java Access Modifiers

```
// public - accessible everywhere
public class PublicClass {
    public String name = "Everyone can see";

    public void publicMethod() { }  // Accessible by anyone
}

// protected - accessible in package + subclasses
class PackageClass {
    protected String name = "Package + subclasses";
    protected void protectedMethod() { }  // Accessible by subclass, package
}

// default (no modifier) - package-private
class DefaultClass {
    String name = "Only in my package";  // No modifier = package-private
    void defaultMethod() { }  // Only same package
```

```
}

// private - accessible in class only
class PrivateClass {
    private String secret = "Only this class";  // Hidden!
    private void privateMethod() { }  // Only this class

    // But private can be accessed by public methods of same class:
    public String getSecret() {
        return secret;  // OK! Same class
    }
}
```

Access Modifiers Visual

```
+-------------------------------------------------------------+
| class MyClass {                                              |
|                                                             |
|     private int a = 1;    <--- Only MyClass can see         |
|     int b = 2;           <--- MyClass + same package        |
|     protected int c = 3; <--- MyClass + package + subclass  |
|     public int d = 4;     <--- EVERYONE                      |
|                                                             |
|     public void method() {                                  |
|         System.out.println(a);  // OK - same class          |
|         System.out.println(b);  // OK - same package        |
|         System.out.println(c);  // OK - same package        |
|         System.out.println(d);  // OK - everywhere          |
|     }                                                       |
| }                                                            |
+-------------------------------------------------------------+
```

Why Private is Protective

```
class BankAccount {
    private double balance;  // Only accessible within this class!

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid amount");
        }
        balance += amount;  // OK - within class
    }

    public double getBalance() {
        return balance;  // Controlled access!
    }
```

```
}

// Outside code CANNOT do:
// account.balance = -1000;  // ERROR! Private!

// Must use:
// account.deposit(-1000);  // But deposit validates!
```

Python Naming Conventions (Analogous to Access Control)

```
class Person:
    def __init__(self, name):
        self.public_name = name      # public - accessible everywhere
        self._protected_name = name # _protected - convention only
        self.__private_name = name   # __private - name mangled

p = Person("Alice")

print(p.public_name)        # Works
print(p._protected_name)    # Works but convention says don't touch
# print(p.__private_name)   # ERROR! Name mangled to _Person__private_name
```

C++ Access Modifiers

```
class Example {
public:          // Accessible everywhere
    int publicField;

protected:       // Accessible in class and subclasses
    int protectedField;

private:         // Accessible in class only
    int privateField;
};
```

JavaScript Private Fields (ES2022+)

```
class Circle {
    // Private field (ES2022)
    #radius = 0;

    constructor(radius) {
        this.#radius = radius;  // Accessible within class
    }

    getRadius() {
```

```
        return this.#radius;  // OK - within class
    }
}

const c = new Circle(5);
console.log(c.radius);  // undefined (not accessible!)
console.log(c.getRadius());  // 5
```

What Should Be Private?

RULE: Make fields private, methods public (usually)

```
What to make PRIVATE:
- Fields (instance variables)
- Helper methods used only internally
- Complex logic that shouldn't be exposed
- Data that could corrupt state if changed directly

What to make PUBLIC:
- Methods that are the "contract" / API
- Getters for read-only access
- Setters only when modification is safe

What to make PROTECTED:
- Methods/fields subclasses might need
- Extension points for subclasses
```

Getter and Setter Pattern

```
class Person {
    private String name;   // PRIVATE - hidden
    private int age;        // PRIVATE - hidden

    // Public getter - READ access
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    // Public setter - CONTROLLED write access
    public void setName(String name) {
        if (name == null || name.isEmpty()) {
            throw new IllegalArgumentException("Invalid name");
        }
        this.name = name;
```

```
    }

    public void setAge(int age) {
        if (age < 0 || age > 150) {
            throw new IllegalArgumentException("Invalid age");
        }
        this.age = age;
    }
}

// Usage
Person p = new Person();
p.setName("Alice");  // Validated!
p.setAge(25);        // Validated!
// p.name = "Bob";   // ERROR! Private!
```

JavaScript Getters/Setters

```
class Person {
    #name;  // Private field

    constructor(name) {
        this.#name = name;
    }

    // Getter
    get name() {
        return this.#name;
    }

    // Setter
    set name(value) {
        if (!value) throw new Error("Name required");
        this.#name = value;
    }
}

const p = new Person("Alice");
console.log(p.name);  // Alice (calls getter)
p.name = "Bob";       // (calls setter)
```

Edge Cases & Traps

Trap 1: Overly Public Fields

```
// BAD! Exposing internal state!
class Point {
```

```
    public int x;  // Anyone can modify directly!
    public int y;
}

Point p = new Point();
p.x = -999999;  // Invalid! No protection!
```

[WARNING] **Professor Trap**: Always use private fields with getters/setters!

Trap 2: Setter That Does Nothing

```
// Useless setter!
public void setName(String name) {
    this.name = name;  // No validation = not better than public field!
}

// Better:
public void setName(String name) {
    if (name == null) throw new IllegalArgumentException();
    this.name = name;
}
```

Trap 3: Getter Returning Mutable Object

```
class Container {
    private List<String> items = new ArrayList<>();

    // DANGER! Returns reference to internal mutable object!
    public List<String> getItems() {
        return items;  // External code can modify!
    }
}

// External code:
container.getItems().add("HACKED!");  // Modifies internal list!

// FIX: Return copy
public List<String> getItems() {
    return new ArrayList<>(items);  // Return a copy!
}
```

[WARNING] **Professor Trap**: Don't return mutable internals! Return copies!

Trap 4: Public Static Field

```
// BAD! Shared mutable state!
public static List<String> sharedList = new ArrayList<>();

// FIX: Private static with accessor
private static final List<String> sharedList = new ArrayList<>();
```

```java
public static List<String> getSharedList() {
    return Collections.unmodifiableList(sharedList);  // Read-only!
}
```

Examples

```java
class BankAccount {
    // Private fields - hidden from outside
    private String accountNumber;
    private double balance;
    private Transaction[] transactions;
    private int transactionCount;

    // Constants
    private static final double OVERDRAFT_FEE = 35.0;
    private static final int MAX_TRANSACTIONS = 100;

    // Constructor
    public BankAccount(String accountNumber, double initialBalance) {
        if (accountNumber == null || accountNumber.isEmpty()) {
            throw new IllegalArgumentException("Invalid account number");
        }
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Initial balance cannot be negative");
        }
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
        this.transactions = new Transaction[MAX_TRANSACTIONS];
        this.transactionCount = 0;
    }

    // Public interface (API)
    public String getAccountNumber() {
        return accountNumber;  // Read-only, no setter
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive");
        }
        balance += amount;
        addTransaction("DEPOSIT", amount);
```

```
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive");
        }
        if (amount > balance) {
            balance -= OVERDRAFT_FEE;
            addTransaction("OVERDRAFT", OVERDRAFT_FEE);
            throw new IllegalStateException("Insufficient funds, overdraft fee charged");
        }
        balance -= amount;
        addTransaction("WITHDRAWAL", amount);
    }

    // Private helper - internal implementation hidden
    private void addTransaction(String type, double amount) {
        if (transactionCount < MAX_TRANSACTIONS) {
            transactions[transactionCount++] = new Transaction(type, amount);
        }
    }
}
```

Example 1: Fully Encapsulated Class

```
class Student {
    private String name;
    private int age;

    // Private constructor - can't use new Student() directly
    private Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Static factory methods - create instances with validation
    public static Student createFreshman(String name) {
        return new Student(name, 18);
    }

    public static Student createAdult(String name, int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Adult must be 18+");
        }
        return new Student(name, age);
    }
```

```
}

// Usage
Student s1 = Student.createFreshman("Alice");  // age=18
Student s2 = Student.createAdult("Bob", 25);    // age=25
// Student s3 = new Student("Charlie", 20);   // ERROR! Private constructor!
```

Example 2: Static Factory Method

```
// In package com.example.models

class InternalData {
    String sensitiveInfo;  // Default (package-private)

    void internalMethod() { }  // Package-private method
}

// This class in same package can access:
class DataProcessor {
    void process() {
        InternalData data = new InternalData();
        data.sensitiveInfo = "Accessible";  // OK - same package
        data.internalMethod();  // OK - same package
    }
}

// But class in different package CANNOT:
class ExternalProcessor {
    void process() {
        InternalData data = new InternalData();
        data.sensitiveInfo = "NOT Accessible";  // ERROR!
        data.internalMethod();  // ERROR!
    }
}
```

Example 3: Package-Level Access

Practice Questions

MCQ 1: Which modifier makes a member accessible only within its class? - A) public - B) protected - C) private - D) default

MCQ 2: Can a static method access an instance variable? - A) Yes, directly - B) Only through an object reference - C) No, never - D) Only if it's final

MCQ 3: What is encapsulation? - A) Hiding implementation details - B) Making fields public - C) Creating multiple constructors - D) Inheriting from a class

Short Answer: Why should fields usually be private? What are the benefits?

Board Coding: Create a class `Temperature` with a private `celsius` field, public `getCelsius()`, `getFahrenheit()`, `setCelsius()`, and `setFahrenheit()` methods. Ensure all values are validated.

Oral Explanation: Explain the difference between public, protected, and private. Give an example of when you would use each.

Solutions

MCQ 1: C) private

MCQ 2: C) No, never - static methods don't have `this` reference

MCQ 3: A) Hiding implementation details

Short Answer: Private fields protect internal state from external corruption. All access goes through controlled public methods (getters/setters) that can validate input, ensuring the object always remains in a valid state. This is the "protective barrier" of encapsulation.

Board Coding:

```
class Temperature {
    private double celsius;

    public Temperature(double celsius) {
        setCelsius(celsius);
    }

    public double getCelsius() {
        return celsius;
    }

    public double getFahrenheit() {
        return celsius * 9/5 + 32;
    }

    public void setCelsius(double celsius) {
        if (celsius < -273.15) {
            throw new IllegalArgumentException("Below absolute zero!");
        }
        this.celsius = celsius;
    }

    public void setFahrenheit(double fahrenheit) {
        setCelsius((fahrenheit - 32) * 5/9);
    }
}
```

Cheat-Sheet

Concept	Key Points
Encapsulation	Bundle data + methods, hide implementation
Private	Accessible within class only
Protected	Accessible in class + package + subclasses
Public	Accessible everywhere
Default	Accessible in package only
Instance member	Each object has its own copy
Static member	One copy shared by all instances
Constructor	Initializes new objects
this()	Constructor chaining (must be first statement)

Review Checklist

- ☐ **Encapsulation**: Can you explain why it's important?
- ☐ **Access modifiers**: Do you know when to use each?
- ☐ **Static vs instance**: Can you explain the difference?
- ☐ **Private fields**: Do you know why to use them?
- ☐ **Getters/setters**: Can you implement proper validation?
- ☐ **Constructor types**: Default, parameterized, overloaded?
- ☐ **this() chaining**: Can you chain constructors properly?
- ☐ **Common traps**: Static accessing instance? Returning mutable? Calling overridable method in constructor?